EcoBarangay Application Review and Recommended Improvements

Based on my review of the uploaded `ecobarangay.zip`, EcoBarangay has the foundation of a comprehensive community environmental management platform. It combines environmental reporting, waste collection schedules, barangay facilities, household registration, eco-points, community engagement, AI assistance, and environmental performance rankings.

My main recommendation is to prioritize security, data integrity, and operational reliability before expanding the application's features. The project already contains many useful modules, but several implementation details need attention before it is suitable for actual use by residents and barangay officials.

1. Overall assessment

Feature coverage

Strong foundation

The code includes resident and official dashboards, maps, reports, community feeds, schedules, rankings, notifications, and AI features.

Security and access control

Critical priority

The supplied Firestore rules allow unrestricted reads and writes, and the API code contains routes that modify data without visible authentication or role checks.

Data management

Needs improvement

The server uses a local JSON file for much of its data while also loading user records from Firestore. Persistence and consistency need to be addressed.

Testing and production readiness

Requires validation

The source code is available for static review, but the ZIP inspection alone does not establish that all workflows build successfully or function correctly in a deployed environment.

Scope: These findings are based on the supplied source files and configuration. They are a code-level review, not a completed penetration test or live deployment test.

2. Critical improvements: security and privacy

These are the highest-priority findings because EcoBarangay may handle residents' names, email addresses, phone numbers, household addresses, environmental complaints, and official records.

1. Restrict Firestore database access

Critical

The supplied `firestore.rules` file grants read and write permission to every document without authentication. This is not appropriate for a production application.

Recommended improvement:

- Require authenticated access where appropriate.
- Apply role-based and barangay-level permissions.
- Prevent residents from modifying other residents' records.
- Restrict sensitive collections to authorized server-side operations.
- Test the rules using Firebase Emulator Suite before deployment.

Do not assume that hiding a screen or button protects its underlying data.

2. Replace the hard-coded official registration password

Critical

The registration endpoint accepts the fixed authorization password `123456` for the `BARANGAY_OFFICIAL` role. Anyone who discovers this value could attempt to register as an official.

Recommended improvement:

- Remove the fixed password.
- Require an administrator-issued invitation or one-time enrollment token.
- Verify the official's identity and barangay assignment.
- Require explicit approval before activating privileged roles.
- Keep role changes exclusively in authorized administrative workflows.

3. Add authentication and authorization to every protected API

Critical

The source shows user-profile updates, report-status changes, facility creation, event creation, schedule creation, and other data-changing endpoints without visible authentication middleware in the route definitions. The admin user-list endpoint also returns user records without a visible authorization check.

Recommended improvement: Verify each request's identity on the server and enforce permissions by role and resource ownership. Residents should not be able to approve reports, alter official records, or award themselves points by modifying API requests.

4. Implement real OTP delivery and secure session management

Critical

The login endpoint logs the OTP and returns a `simulatedOtpCode` to the client. The password-reset endpoint similarly exposes the generated reset code in its response. This defeats the purpose of independent email verification in a production system.

Recommended improvement:

- Send OTPs through a real email delivery service.
- Never return OTP values in API responses or write them to logs.
- Issue a secure, expiring session after successful verification.
- Add rate limits to login, resend-code, and password-reset endpoints.
- Invalidate outstanding sessions after password changes or account suspension.
- Make password-reset codes single-use and enforce attempt limits.

5. Protect sensitive user data in API responses

The user lookup endpoint returns the stored user object directly, while profile updates pass the request body to the data layer. These patterns warrant checking for exposure of password hashes, household information, and unauthorized field changes.

Recommended improvement: Use explicit response schemas that exclude credentials and private fields. Accept only approved profile fields, validate every update, and limit household-address visibility to authorized purposes.

3. Improve the database and reliability

The project has a `dbStore` layer that reads and writes application data to a local `data/db.json` file, and it also loads user records from Firestore.

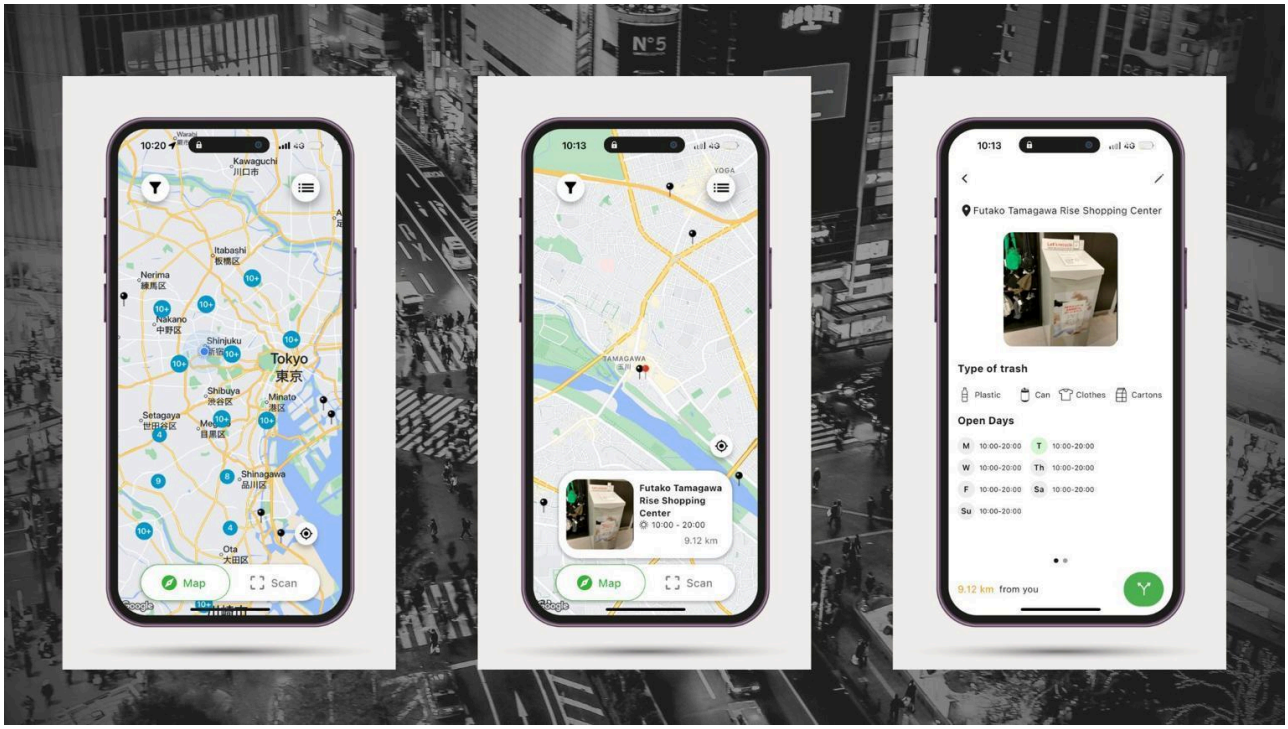
This arrangement can be useful for a prototype, but it creates risks when several users access the application concurrently or when the application is deployed to infrastructure with ephemeral local storage.

Issue	Recommended improvement
Split data storage	Select one authoritative database for operational records.
Concurrent updates	Use transactions or atomic operations for points, votes, and report statuses.
Data loss after restart or redeployment	Store persistent data in a managed database with backups.
Duplicate or inconsistent records	Establish unique identifiers, constraints, and validation rules.
Incomplete records	Define required fields and data types for each entity.
Difficult recovery	Implement scheduled backups and tested restoration procedures.
Limited traceability	Record who changed a record, what changed, and when.

Priority recommendation: Standardize the database architecture before migrating real household or environmental data. If Firebase is retained, use its transactional capabilities and server-side authorization; if another database is selected, migrate all operational modules consistently rather than maintaining competing sources of truth.

4. Improve the application's environmental management features

EcoBarangay's strongest opportunity is to become more than an environmental information portal: it can support a complete, traceable workflow from resident reporting to barangay action and measurable environmental outcomes.



Add to Favorites

A. Environmental reporting and complaint resolution

Improve the reporting module with:

- Report categories such as illegal dumping, missed collection, clogged drainage, and improper waste disposal.
- Photo evidence, timestamps, location, and unique tracking numbers.
- Status progression: Submitted → Under Review → Assigned → In Progress → Resolved.
- Resolution evidence and confirmation by an authorized official.
- Duplicate-report detection and a resident feedback mechanism.

Success measure: Median resolution time, percentage resolved within target, and percentage of reports reopened.

Recycle & Garbage Collection Schedule
2026

Garbage is picked up *weekly* and Recycling is picked up *every other week*. Every household is assigned a day for collection and a corresponding Red or Blue color zone for Recycling. If you are unsure of your collection day or color zone, please refer to the instructions at the top of the Collection Map page.

The calendar below indicates the recycle (red/blue) weeks.

The holidays indicated with diagonal stripes will apply to garbage and recycle collection.

Trash Collection Key:

- Normal Days
- Rescheduled Days
- Alternate Days



Garbage (336) 222-5111 Recycling (336) 229-0525 BurlingtonNC.gov/SolidWaste

Add to Favorites

B. Waste collection scheduling

Strengthen the schedule and bulk-pickup modules with:

- Collection calendars by barangay, zone, and waste type.
- Separate schedules for biodegradable, recyclable, residual, and special waste.
- Notifications for schedule changes, missed collections, and pickup confirmations.
- Assignment of collection teams and vehicles.
- A history of completed, missed, and rescheduled pickups.

Success measure: On-time collection rate, missed pickups, and average time to complete bulk-pickup requests.



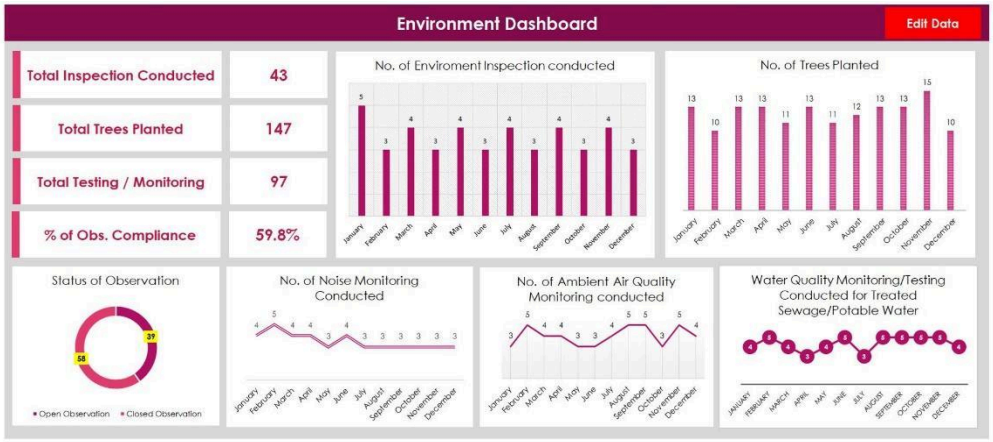
Add to Favorites

C. Household registration and eco-points

Make the rewards system transparent and resistant to manipulation:

- Record the reason and evidence for every point transaction.
- Prevent repeated rewards for the same activity.
- Require validation of reported waste weights and completed activities.
- Add an audit trail for point adjustments.
- Publish clear rules for earning and redeeming points.

Success measure: Verified participation, household segregation compliance, repeat participation, and validated kilograms of waste diverted from disposal.



Add to Favorites

D. Barangay environmental rankings

Ensure that rankings are based on transparent, comparable, and current indicators:

- Document the formula and weight of each indicator.
- Distinguish verified data from estimates and self-reported figures.
- Show the reporting period and date of the latest update.
- Normalize indicators when barangays differ substantially in population or service coverage.
- Show missing data rather than automatically treating it as zero.
- Preserve historical rankings so users can track progress over time.

Success measure: Data completeness, verification rate, and improvement in environmental indicators over comparable reporting periods.

5. Improve the AI features

The source includes an AI photo-validation endpoint and an EcoBarangay assistant powered by Gemini. These are useful additions, but their outputs should support—not replace—human decisions.

Feature	Improvement
Photo validation	Return a confidence score, detected issue, and reason for flagging.
False detections	Allow authorized officials to accept, reject, or correct AI suggestions.
Environmental chatbot	Base answers on approved local waste policies and collection schedules.
Incorrect AI answers	Display uncertainty and direct users to official sources where appropriate.
Cost and misuse	Limit request sizes, apply rate limits, and monitor AI usage and expenditure.
Privacy	Minimize personal information sent to the AI service and define retention rules.
Evaluation	Test accuracy using a labeled set of genuine and non-genuine environmental reports.

A key distinction is that an AI-generated assessment should not automatically count as verified evidence. For example, a photo classifier might identify a pile of waste, but it cannot reliably establish when the waste was deposited, who was responsible, or whether a complaint has been resolved.

6. Improve the user interface and accessibility

The application contains resident and official dashboards, navigation components, a map, and several modal workflows. The next step is to ensure these components offer consistent, accessible, and easy-to-understand experiences.

Mobile-first design

Optimize report submission, camera uploads, map interaction, and collection schedules for residents using mobile phones and slower internet connections.

Accessibility and language

Check keyboard navigation, screen-reader labels, contrast, text scaling, and clear error messages. Offer Filipino alongside English, and consider additional local languages based on the deployment area.

Actionable notifications

Give users meaningful updates about report status, collection schedules, events, and pickup requests. Allow notification preferences and prevent duplicate alerts.

Map usability

Provide a list alternative to the interactive map, useful filters, accessible location descriptions, and clear handling of unavailable or inaccurate coordinates.

Consistent workflows

Standardize loading indicators, confirmation dialogs, validation messages, empty states, and error recovery across all modules.

7. Improve maintainability and deployment

The project uses React, TypeScript, Express, Firebase, and Vite. Its existing structure is a reasonable foundation, but the large application and data-store modules would benefit from clearer separation of responsibilities.

Area	Recommended action
API architecture	Separate route handlers, business logic, authorization, and database access.
Input validation	Use typed request schemas and reject invalid or unexpected fields.
Error handling	Centralize API errors and avoid returning internal exception details to clients.
File uploads	Validate file type, actual file content, and size; restrict storage access.
Logging	Add structured logs while excluding passwords, OTPs, tokens, and unnecessary personal data.
Dependencies	Review outdated or vulnerable packages and lockfile consistency.
Build quality	Run TypeScript checks, production builds, and automated tests in CI.
Deployment	Document environment variables, database setup, migrations, backups, and rollback procedures.

One additional concern is that `server.ts` configures a JSON request-body limit of 10 MB, and the AI image endpoint accepts image data through that API.

Image-specific limits and validation should be applied rather than relying only on the general request limit.

8. Recommended implementation roadmap

I recommend implementing the improvements in the following order.

Phase 1 — Security and data protection

Highest priority

Remove public database permissions, eliminate the hard-coded official password, secure authentication and OTP delivery, protect API routes, and prevent unauthorized user-data access.

Exit criterion: Unauthorized users cannot read protected records, change roles, or perform official actions.

Phase 2 — Database and reliability

Choose the authoritative database, implement transactions and validation, establish backups, and test data recovery.

Exit criterion: Concurrent updates are consistent and records survive restarts and redeployments.

Phase 3 — Complete environmental workflows

Improve reporting, complaint resolution, collection scheduling, household rewards, notifications, and audit trails.

Exit criterion: Residents and officials can complete and track the full lifecycle of each major activity.

Phase 4 — AI, analytics, and usability

Evaluate AI performance, improve accessibility and mobile usability, and strengthen environmental dashboards and rankings.

Exit criterion: AI outputs are evaluated, indicators are traceable, and users can navigate essential workflows reliably.

Phase 5 — Testing and pilot deployment

Run security tests, integration tests, usability sessions, backup restoration, and a controlled pilot in selected barangays.

Exit criterion: Critical defects are resolved, access controls are verified, and operational staff are trained.

9. Suggested testing and evaluation framework

Before accepting the system for operational use, test it against measurable criteria.

Test category	Example acceptance criterion
Authentication	Invalid credentials and expired OTPs never establish a session.
Authorization	Residents cannot access another user's private records or invoke official-only operations.
Data integrity	Duplicate submissions and concurrent point updates do not produce inconsistent records or rewards.

Test category	Example acceptance criterion
Environmental reporting	Each report has a traceable status history and an authorized resolution record.
Scheduling	Schedule changes appear consistently to affected residents and officials.
AI validation	Precision, recall, false-positive rate, and false-negative rate are measured against labeled examples.
Accessibility	Core workflows can be completed using keyboard navigation and assistive technology.
Performance	Response times and error rates meet defined targets under realistic concurrent usage.
Recovery	Backups can be restored and validated within documented recovery targets.

These are proposed acceptance criteria, not claims that the application has already passed these tests.

10. Final recommendation

EcoBarangay has considerable potential as a digital environmental governance platform for Philippine barangays. Its combination of community reporting, waste management, citizen participation, AI assistance, and environmental performance tracking can support both operational improvements and research on digital local governance.

I would not recommend deploying the current version with real residents' sensitive information until the security and authorization issues are resolved.

The most valuable next step is a focused technical improvement of the existing codebase, preserving the current modules while fixing the high-risk issues first. Once those are addressed, the application can be evaluated through a controlled barangay pilot, with evidence collected on usability, response times, resident participation, and measurable environmental outcomes.